AI WEB 开发挑战赛 · 技术文档

Margin
研发辅助 Agent 工作台

一个工作台 + 多个可插拔 AI 研发助手
代码审查 · 架构设计

赛题方向 7 · AI 代码开发辅助

参赛组别：B 组

学校：百色学院

团队成员：黄维涛（队长）、黄建康

指导教师：蒋锐

联系方式：3444203102@qq.com

交付版本：见仓库 Git 提交历史 · 仓库地址：github.com/15667h/Code-review-agent

目录

1 **项目概述** 1.1 一句话说明 · 1.2 要解决的问题 · 1.3 产品形态 · 1.4 与同类工具的差异 · 1.5 交付规模

2 **需求分析** 2.1 目标用户与场景 · 2.2 功能需求 · 2.3 非功能需求 · 2.4 赛题评分维度映射

3 **系统架构设计** 3.1 总体架构 · 3.2 Agent = Model + Harness · 3.3 前端工作台
3.4 事件流 · 3.5 关键架构决策 · 3.6 技术选型理由

4 **AI 技术方案** 4.1 模型集成 · 4.2 Agent 循环 · 4.3 Prompt 工程 · 4.4 工具接口设计
4.5 上下文预算与压缩 · 4.6 缓存感知的上下文结构
4.7 安全与可控性 · 4.8 规则与模型的交叉验证 · 4.9 结构化产出
4.10 三层质量约束：确定性核验 → 独立判官 → 证据审查

5 **核心功能实现** 5.1 代码审查助手 · 5.2 架构设计助手 · 5.3 增量审查
5.4 多助手隔离 · 5.5 账号与会话安全 · 5.6 前端设计系统

6 **测试与部署** 6.1 测试体系 · 6.2 评估体系 · 6.3 架构助手难度实测
6.4 与业界水位对照 · 6.5 部署与运行

7 **创新点说明** 7.1 技术创新点 · 7.2 应用创新点

8 **总结与展望** 8.1 已交付能力 · 8.2 已知边界与不足 · 8.3 后续计划

1 项目概述

1.1 一句话说明

Margin 是一个面向研发流程的 AI 工作台：它把「读代码、找问题、讲结构」这类需要长时间专注的工作交给一组可插拔的 AI 助手，并用一套统一的界面承载它们的产出。 当前内置两个助手 —— **代码审查**（产出带证据的发现列表）与 **架构设计**（产出可评审的分层蓝图）。

1.2 要解决的问题

研发过程中有两类高价值但耗时的智力劳动，长期依赖资深工程师的时间：

问题	传统做法的困难	本项目的做法
代码审查 找缺陷、看资源生命周期、查边界条件	人工逐文件阅读，成本高且不稳定；简单静态扫描工具只能覆盖规则化问题，误报多、无法理解语义	让 Agent 带着工具循环读代码（读文件、搜符号、看 diff、跑规则）， 产出的每条发现都必须带**文件与行号证据**，并按严重级归类
架构理解与设计 接手陌生仓库、评审分层与依赖	需要通读大量代码才能画出结构图；口头结论无法沉淀，评审时缺少可讨论的载体	让只读的架构助手产出**分层图 + 组件清单 + 关键决策 + 开放问题**， 每条决策写明「放弃了什么、代价是什么」，形成可评审的方案草稿

1.3 产品形态

界面是一间「工作室」： 左侧是助手轨道与空间（被审仓库）列表，中间是工作台（对话 + 产出）， 下方是输入坞。切换助手时，不只是换一个图标 —— **产出形态、工具集、界面文案、强调色** 整套都跟着变。



图 1-1 工作台骨架：轨道 + 上下文栏 + 工作台 + 输入坞

1.4 与同类工具的差异

维度	常见的 AI 编程工具	Margin 的做法
定位	编辑器内的代码补全 / 对话	独立工作台，承载**完整的一次审查或设计任务**及其产出
产出	自由文本回复，需人工再整理	**结构化产出**：发现 / 组件 / 决策 / 开放问题各有 schema，前端按形态渲染
证据	可能给出不存在的行号	**三层核验**：位置真实性 → 独立判官复核 → 证据与权衡审查（详见 4.10）
扩展	新能力需要改代码	**助手是一份声明**（AgentSpec），接入新助手不改既有代码
可信度	难以评估效果	自带评估链路：12 个靶场 / 134 条标注 / 双口径指标 / 人工复核，数字由脚本现算
模型接入	绑定特定厂商	只填**接口地址 / Key / 模型名**三项即可接任何 OpenAI 兼容服务

1.5 交付规模

29,453
行源码（前端 + 后端 + 测试 + 评估）

282
项回归测试（全绿）

2
个内置 AI 助手（可插拔扩展）

1
个外部运行时依赖（httpx）

组成部分	文件	行数	说明
Agent 运行时（harness）	26	10,609	主循环、上下文、工具、护栏、三层核验、HTTP 服务
回归测试	13	4,373	含前端契约测试与 SSE 集成测试
评估链路	8	2,253	靶场运行、指标计算、报告生成
前端脚本 / 样式 / 页面	16	12,052	原生 JS + CSS，零构建、零 npm 依赖
仓库卫生检查	1	166	死代码与未使用导入扫描

2 需求分析

2.1 目标用户与场景

用户	典型场景	他真正想要的
接手陌生仓库的工程师	入职新项目、接手遗留系统	先看清分层与依赖，再决定从哪读起 —— 而不是从第一个文件顺序读下去
提交前的作者	自查本次改动、准备 PR	只审自己的改动，别把全仓旧问题一次性倒出来
做评审的资深工程师	评审分层方案、技术选型	可讨论的载体：决策写了代价、不确定的单独列出，能逐条反驳
教学与实训	讲解代码缺陷模式	每条发现都指向具体行，可当场打开核对

2.2 功能需求

编号	需求	验收判据	实现位置
F1	代码审查	产出结构化的发现列表，每条含文件、行号、严重级、复现证据与修复建议	助手 review
F2	架构设计	产出分层、组件与依赖、决策（含代价）、开放问题	助手 architect
F3	助手可插拔	新增助手只需一份声明；前端按清单渲染，不改代码	agents.py / /api/agents
F4	运行可追溯	每次运行的每一轮工具调用、耗时、用量都可回放	SSE 事件流 + 归档
F5	增量审查	只审 BASE..工作树的改动，并反查改动之外的调用点	diffmode.py
F6	产出可核验	每条产出都能给出「凭什么这么说」的代码位置，且位置可机械核对	三层核验
F7	模型可更换	只填三项即可接任意 OpenAI 兼容服务	config.py
F8	离线可用	无 API Key 时仍能跑完整链路（规则驱动）	offline.py

2.3 非功能需求

维度	要求	本项目的做法
可审计性	安全类工具本身必须可读、可审查	零第三方依赖（仅 httpx）；HTTP/SSE 用标准库手写
安全性	路径不得逃逸、命令受限、密钥不入库	工作区解析校验 + 命令白名单 + .env 隔离（有回归测试守着）
成本	长任务不能烧钱	静态前缀恒定，KV 缓存命中率实测约 87%
可恢复性	中断后不能留下半份产出冒充完整	停止时如实标注「已停止，上面是停止前已完成的部分」
诚实性	失败不能说成空结果	工具层显式失败信封；前端连不上后端时明确说未连接

2.4 赛题评分维度映射

评分维度	本项目的对应交付	详见
技术创新性	三层质量约束（确定性核验 / 独立判官 / 证据审查）；助手即声明；缓存感知上下文	第 7 章
AI 技术应用深度	结构化产出 + 工具协议 + 渐进披露 + 判官复核 + 判据可验证	第 4 章
工程完成度	29,453 行、282 项回归、12 靶场评估、可插拔架构	第 5、6 章
可用性与体验	明暗双主题工作台、会话隔离、增量模式、失败可读	5.6 节
文档与规范	README / 技术文档 / 贡献指南 / 许可证 / 评估报告（脚本生成）	全篇

3 系统架构设计

整个系统的设计围绕一条主线：**把 AI 的不确定性关进工程结构里。** 下面按「分层 → 抽象 → 界面 → 事件 → 决策」展开，每条关键决策都注明替代方案与代价。

3.1 总体架构

系统分为四层：浏览器端工作台、HTTP/SSE 服务层、Agent 运行时（Harness）、以及被 Agent 调用的工具与沙箱。 其中**工具层是 Agent 与外部世界（文件系统、命令、模型）之间的唯一通路** —— 所有读写都经过它，因此安全护栏只需在这一层收口。





图 3-1 总体架构：四层 + 唯一通路原则

3.2 Agent = Model + Harness

项目的核心抽象来自一个判断：**模型本身不可编程，能编程的是它外面的那层骨架。** 把这层骨架（Harness）设计好，同一个模型就能稳定地完成不同任务。 Harness 由五部分组成：

组成	职责	本项目的实现
上下文管理	决定模型每一轮看到什么：静态前缀 + 动态追加	系统提示与核心工具定义恒定；进度、状态、检索到的工具 schema 追加到轨迹末尾
工具接口	把能力以模型能理解的方式暴露	JSON Schema 参数；描述写「何时用 / 何时不用」；数组参数显式声明
约束	限制行为边界，防止失控	迭代上限、重复调用检测、连续被拒熔断、命令白名单、只读助手硬契约
验证	检查产出是否可信	三层核验（位置 → 判官 → 证据与权衡）+ 确定性规则交叉验证
纠正	出错后给出可操作的修复路径	错误信息四层（类型 + 参数 + 调用栈 + 建议）；失败显式而非伪装成空结果

3.3 前端工作台

前端不做「一个助手一套页面」，而是**一套骨架 + 声明驱动的差异**：

- **骨架共享**：助手轨道 + 上下文栏 + 工作台 + 输入坞，所有助手走这套结构
- **差异来自声明**：助手清单由后端 `GET /api/agents` 下发， 前端据 `outcome` 选产出渲染器、据 `capabilities` 决定显示哪些入口 （例如只有声明了 diff 能力的助手才出现「增量」开关）
- **强调色只是声明里的一个色值**： `body[data-agent]` 注入 `--agent`， 深浅三档由 `color-mix` 派生 —— 接新助手只换一个色值
- **兜底渲染器**：不认识的产出形态按字段原样列出，不白屏、也不假装它是发现列表

3.4 事件流：一条路径同时服务「实时」与「重放」

运行过程中的每一步（开始、决策、工具调用、产出、重试、压缩、完成）都以事件形式追加到运行记录。 SSE 连接**先补发历史事件、再实时推送**，因此「打开一条正在跑的运行」与「回放一条已完成的运行」 走的是同一条

路径 —— 不存在两套渲染实现。

运行产生事件 → 追加到记录（带序号） → SSE 补发 since 之后的事件 → 实时推送新事件 → **前端 applyEvent**

3.5 关键架构决策

决策	替代方案与为什么不选	代价
标准库实现 HTTP 层	FastAPI/Flask 更省事，但引入依赖链；安全类工具需要可审计	需手写路由、分块传输、SSE 终止序列（都踩过坑）
静态前缀恒定	把时间/进度写进系统提示更直观，但会让 KV 缓存整轮失效	动态信息必须另想办法追加（状态栏）
产出落类型化桶	统一用自由文本更灵活，但无法按形态渲染	每加一种形态要写 schema 与渲染器
未知助手直接报错	回退到默认助手更「健壮」，但会产出答非所问的结果	调用方要处理 400
判官只删不加	让判官也能补充会更「完整」，但那等于把审查者与被审者合成一个角色	漏报无法由判官补回（可接受：宁可漏也不加噪声）

3.6 技术选型理由

- Python 3.12：`ast` 标准库足以做结构化代码分析，无需引入解析库
- **httpx 作为唯一运行时依赖**：同步调用 + 超时 + 连接池，比 urllib 更可控
- **原生 JS/CSS 前端**：无构建、无 npm 依赖，评审者可 clone 即读；对工具类产品， 这比框架带来的开发效率更重要
- **OpenAI 兼容协议**：业界事实标准，换供应商只改三个值

4 AI 技术方案

本章是文档的重点：一个能跑起来的 AI 应用与一个可靠的 AI 应用之间的差距， 几乎全部落在这里 —— 上下文怎么组织、工具怎么设计、失控怎么兜住、产出怎么验证。

4.1 模型集成

4.1.1 集成方式

模型调用集中在单一客户端模块（`harness/model.py`），对上层暴露「给消息与工具，返回一次决策」的接口。这样重试策略、退避、用量统计、 错误分类只写一遍，Agent 循环里不掺协议细节。

能力	实现要点
失败重试	只对可重试错误（限流、网络、5xx）重试；指数退避；重试以事件推给界面（用户看到「在重试」而不是卡住）
用量统计	累计输入 / 输出 / 缓存命中 token，随运行结果归档 —— 成本可回溯
工具调用	走 function calling 通道；参数解析失败时归类为参数畸形并给出重试指引
降级	没有 API Key 时切到启发式模型：仍走完整 Agent 循环与工具协议，产出由确定性规则给出

4.1.2 只填三项的接入设计

协议固定为 OpenAI 兼容，因此配置只需三项：**接口地址、API Key、模型名称。** 其余差异都能从这三个值推出来，不需要用户选「提供商」。

真实操作	系统如何处理
贴完整请求地址 `https://x/v1/chat/completions`	自动裁成根地址。不裁会请求 `/chat/completions/chat/completions` —— 一个会被误判成「服务挂了」的 404
只写域名 `api.x.cn/v1`	按公网 https / 本机内网 http 自动补协议
自建服务用任意 Key（vLLM / Ollama）	不强求 `sk-` 前缀：前缀不是任何协议的要求，强求等于把合法配置挡在门外
模型名写错	测试连接返回「接口可达，但列表里没有这个模型，可用：…」，并**不判失败** —— 有些服务不实现 `/models`

4.2 Agent 循环设计

组装上下文 → 请求模型决策 → 执行工具调用 → 观察结果入轨迹 ↺ **收尾（三层核验）**

4.2.1 规划：把「打算怎么做」显式化

助手被要求用任务清单工具（`update_todo`）写下计划并更新状态。 显式计划让「读了三个文件就下结论」这种偷懒行为变得可见，也让用户能中途判断方向。

4.2.2 工具调用：能力按需可见

工具集分两档：**核心工具**进静态前缀，随叫随到；**扩展工具**需先检索再加载 schema。 这解决两个实际问题：上下文占用（二十多个工具定义全量前置会吃掉可观预算）、 选择质量（候选越少选错越少）。

4.2.3 收敛：轮次预算

实测发现「撞上限」的根因不是能力不足，而是**模型不知道自己还剩几轮。** 因此每轮把进度写进状态栏，按**剩余比例**分三档：

剩余比例	提示强度	理由
> 40%	只报数字	不打扰，避免过早收尾
20%–40%	「可以开始收尾」	剩余轮次优先留给补证据与写代价，而不是继续扩大阅读
≤ 20%	「立刻 finish，未覆盖的写进开放问题」	半份产出比明确声明「未完成」更糟 —— 评审会以为那是全部

> **踩过的坑（真实记录）。**早期熔断用「累计被拒次数」判定，一旦打开就再也不会关闭， 模型被自己的护栏锁死 —— 连 `finish` 都被拒绝。 现在改成判据是**连续**被拒次数（成功一次即归零），且收尾与产出类工具走前缀豁免。 教训：**护栏必须可恢复**，否则它就是故障源本身。

4.3 Prompt 工程

4.3.1 分层结构

层	内容	为什么放这里
身份与目标	你最擅长什么；本次任务的判据	最先进入上下文，决定整体基调
工具契约	核心工具定义（何时用 / 何时不用）	恒定不变，可被缓存
硬约束	只读边界、证据要求、不得编造行号	紧跟在目标之后，避免被任务细节冲淡
失败兜底	读不到文件怎么办、路径不存在怎么办	没有兜底规则时模型会即兴发挥，通常发挥得不好
风格与格式	产出字段要求、语言	放最后，不占早期注意力

4.3.2 工具描述的三条规矩

- **必须写「何时不用」**：只写能干什么，模型会在不合适的场景使用它
- **数组参数要在描述里明说**：schema 声明 `type: array` 不够 —— 实测模型连续 5 次把数组字段填成字符串。描述必须写「数组，每项一个…」
- **错误信息给出目标形态**：不只是「类型错误」，而是「即使只有一项也要写成 `['…']`」

4.3.3 用「照妖镜」防背诵式作答

架构助手最典型的失败是「背架构图」：把教科书里的通用结构套上去，而不是读代码。 因此在任务说明里直接点出该仓库的照妖镜，例如对 requests：

> requests 本身**不实现重试**，重试在 urllib3 的 `Retry`。 如果你的方案里出现了「重试层」，说明你没读代码。

评估时这条会被机器检查：参考蓝图里声明了 `must_not_contain`， 命中的组件会被单独告警 —— 这类「多报了一个不该有的东西」靠精确率抓不住重点。

4.4 工具接口设计

工具	作用	设计要点
`list_files`	列目录	目录不存在显式失败（不是「没有匹配」）；跳过点目录与依赖目录
`read_file`	读文件（带行号）	支持分段；行号前缀为证据引用服务；offset 越界显式报错
`grep`	正则搜索	非法正则给转义提示；命中上限提示「结果不完整」而不静默截断
`read_symbols`	符号表（函数/类清单）	建立结构认知的高性价比手段，避免通读全文
`sketch`	一次 ast 扫描出全仓结构	两阶段的第一步：先拿结构再判断。逐文件读会读完之前用完轮次预算
`git_diff`	看变更	审查场景的起点：只看改动比通读全仓更有效率
`discover_tools`	检索扩展工具	渐进披露的入口；按需求检索而非全量列举
`record_*`	登记产出（发现/层/组件/决策/问题）	边确认边登记；同名去重；数组参数在描述里明说
`update_todo`	任务清单	让计划可见、可纠偏
`finish`	收尾	必须写范围与理由；统计口径随产出形态变化

4.5 上下文预算与压缩

- **预算对象**统计静态前缀、历史轨迹与工具输出的 token 占用
- **压缩触发**：占用达窗口 80% 时，对较早的工具输出做摘要化，保留结论与证据，丢弃大段原文
- **截断提示**：单个输出过大时落盘并在上下文里只给摘要与路径，同时明确标注「已截断」，避免模型以为拿到了全部
- **统一硬上限**：读文件、搜索、列表全部有显式上限，且**上限生效时必然告知** —— 静默截断会让模型得出「全仓都没有」的假结论

4.6 缓存感知的上下文结构

这是成本与延迟上的关键优化，做法简单但需要纪律维持：

恒定的静态前缀
系统提示 + 核心工具定义，顺序与内容**永不变化。**任何动态内容（时间、进度、状态、 按需加载的工具 schema）都追加到轨迹末尾，绝不插进前缀。

为什么有效
主流模型服务按「前缀相同的最长部分」计费与加速。前缀稳定时，每轮只有尾部新增部分 需要重新计算 —— 本项目实测 KV 缓存命中约 87%。

维持这条纪律的难点。「把当前时间放进系统提示」是很自然的写法，但它会让前缀每轮都变， 缓存全部失效。因此这类信息统一由「运行时状态栏」追加在轨迹尾部， 并以自检方式暴露前缀指纹，防止后续改动无意破坏这个不变量。

4.7 AI 安全与可控性

失控模式	护栏	实现要点
死循环	迭代上限 + 重复调用检测	连续相同工具与参数 N 次即判定陷入循环，强制收尾
越界读写	路径围栏	所有路径 resolve 后校验必须位于工作区内；符号名也做同样校验
危险命令	命令白名单 + 风险分级	不在白名单的命令直接拒绝；高风险工具需显式开关
密钥外泄	路径拦截	`.env`、私钥、凭据文件在读取层被拦下
声明与能力脱钩	只读硬契约	声明只读的助手若注册写工具，**运行会在启动时直接失败**，而不是靠自觉
静默失败	显式失败信封	错误含类型、参数、调用栈与建议四层

4.8 规则与模型的交叉验证

「能确定性判定的就不交给模型」—— 规则集既独立产出，又作为模型结果的验证器： 规则命中而模型未报标为疑似漏报，两者都报则提高置信度。 实测中这一机制把规则难以覆盖的语义类缺陷与模型容易忽略的机械模式各自补齐。

4.9 结构化产出

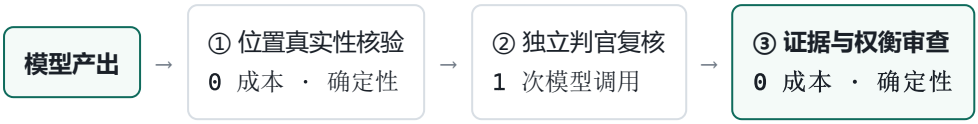
产出不是自由文本，而是有 schema 的结构化数据，落进按类型分桶的容器：

助手	产出形态	字段
代码审查	`findings`	severity · file · line · title · detail · evidence · fix · diff
架构设计	`blueprint`	layer（名称/职责/证据）· component（职责/依赖/失效模式/证据）· decision（决策/备选/代价/证据）· open_question（问题/影响/是否阻塞）

结构化带来三个直接好处：前端能按形态渲染；产出能被机械核验（见 4.10）； 评估可以逐条比对答案键。

4.10 三层质量约束：确定性核验 → 独立判官 → 证据审查

本节是本项目在 AI 可靠性上的核心工作。三层按**成本从低到高**排列，前一层的输出是后一层的输入 —— 先用零成本的方式把明显站不住的挡掉，再花钱让模型复核剩下的。



4.10.1 第一层：位置真实性核验（零成本，确定性）

最初的 `verify_citations` 只核对「引用的文件读过没有」，不核对 **「引用的那一行是否真的在支撑这条描述」**。这正是「看似专业但其实错」的软肋。 现在直接读真实文件判断，三条判据（任一成立即通过）：

1. 引用行里出现了描述中的技术线索（关键词族，如 shell / 加密 / 资源释放 / 越权）
2. 引用行邻近上下文里出现了该线索（缺陷常写跨多行表达式）
3. 引用行含描述提到的标识符（函数名、变量名）

行号越界（文件被改写或纯编造）、文件不存在、指向空行或纯注释，一律判不可信。 这条判据是**实测出来的，不是想出来的**：

> **三轮修正记录。**第一版对 4 条人工构造的假发现只拦住 1 条 —— 原因是关键词族里放进了裸的「query」「拼接」，它们也出现在无关描述里。 收紧后仍有 1 条漏过（指向空行，却靠邻域里的线索蒙混过关）。 最终判据改为「线索命中**且**引用行本身不是结构行」才 4/4 全部拒绝， 同时保证 20 条真实发现零误杀。

4.10.2 第二层：独立判官（只删不加）

运行收尾前，把每条发现连同它引用的**真实代码（带行号）**交给一次独立判断： 只问「这段代码能证明这个结论吗」，答 drop 或 keep。

设计约束	理由
只删不加	让判官既能删又能加，等于把审查者与被审者合成一个角色 —— 正是共享盲区
先做位置核验再送判官	位置站不住的发现不进判官：省 token，也避免对着无关代码猜
失败即保留全部	判官是降低误报的增强项，不是正确性前提。它挂了就清空结果，会把一次网络抖动放大成「什么都没查到」
输出不可解析也保留全部	漏判不等于判错

实测效果：在 `vulnerable-service` 上，两条「结论与代码相反」的假发现被准确剔除， 并给出具体理由（「第 22 行已是 `max(0, ...)`，已有下界保护」）； 20 条真实发现零误杀，耗时 7.3 秒。

> **一个必须记下的发现：判官口径与标注口径并不相同。** 在边界条件靶场上，判官删掉了两条真实缺陷，理由是「`LIMIT - hits` 返回负数在数学上是对的」「闭区间是设计选择」。判官按**「代码是否写错」**判断，而标注按**「前提是否被约束」**判断 —— 两者不同维度。于是加了一道确定性闸：低置信度提示只有在判官明确指出「代码里没有这回事」时才允许删，其余情况保留。修正后召回从 80% 回到 86%，精确率不变。

4.10.3 第三层：证据与权衡审查（零成本，确定性）

架构产出的性质与缺陷不同：它说的是「这个层 / 组件存在于这里」，关键词族对不上。 因此换判据，但沿用同一个原则 —— **能确定性判定的绝不交给模型。**

证据核验

四类产出（层 / 组件 / 决策 / 开放问题）都带 `evidence` 字段，格式是完整相对路径加行号。 三档核验结果：

状态	含义	处理
ok	路径存在、行号在范围内、指向代码而非空行/注释	计入「可精确定位」
weak	只有文件名（需补全路径），或指向空行与纯注释	提示补全，不判死 —— 有些架构事实确实由注释声明
bad	文件不存在或行号越界	这条产出没有依据

「没给证据」与「证据是错的」**分开统计**：前者是遗漏，后者是编造，性质不同。

权衡真实性

业界公认最难识别的失败模式：模型能为任何选择写出流畅理由，把「单纯的偏好」包装成 「严谨的权衡」—— 表面过关，底下没通过。两条确定性规则逼近它：

1. `consequences` 必须含负面信号；只有好处没有代价的选择，通常不是选择，而是还没想清楚
2. `alternatives` 里至少有一条写出了**它自己的优点** —— 只有缺点的备选是稻草人

> **这条检查也被真实产出打过脸。**初版只查「成本 / 风险 / 复杂度」这类名词， 结果在 requests 案例上把 4 条写得很好的决策全判成「只写好处」。原文是这样的： 「依赖图里是假边：只在类型检查期存在，运行时不存在」「必须穷举 urllib3 各版本的异常类型」 「多线程共享同一 Session 时无锁保护」—— 一句「成本」都没有，但都是实打实的代价。 于是改成三条件联合（没有负面词 **且** 没有转折结构 **且** 短到不像认真写过）， 并把稻草人判据从「逐条备选都要有优点」改成「整条决策至少一条有优点」。 修正后三案 21 条决策零误报，同时四类真问题仍全部能抓到。

4.10.4 三层合起来的效果

指标	改造前	改造后
架构证据可精确定位率	9%（55 处引用仅 5 处写全路径）	**95%–99%**（三案例实测）
编造/错位引用	未检测	0（bad 计数为 0）
小靶场精确率	76%	76%（小样本本就没有多少误报可删）
小靶场召回	86%	86%（加低置信度保护前会掉到 80%）
架构依赖边 F1	—	19%–88%（按案例差异大，与参考蓝图完整度强相关）

> **诚实结论**：三层核验在**大样本**上收益明显（证据可核验率从 9% 提到 95% 以上）， 在**小样本**上几乎无收益（小样本误报本来就少），但会带来误删风险 —— 所以低置信度保护是必要防线。 不夸大它，也不因为「没测出提升」就撤掉它。

5 核心功能实现

5.1 代码审查助手

一次审查的完整链路：

选仓库 → 描述重点 / 勾选范围 → 运行（可实时观察） → 三层核验 → **发现列表 + 可回放**

每条发现都带：严重级别、文件与行号、复现证据、修复建议，若涉及代码改动还会给出建议 diff。 列表支持按严重级别筛选，计数随产出实时增长。

5.2 架构设计助手

架构助手是**只读**的（`can_edit=False`），工具集里没有写工具 —— 这是硬契约，声明与能力不允许脱钩。产出为分层蓝图：

产出	要求
分层	每层说清「负责什么」与「**不负责什么**」；层数宜少不宜多，超过 5 层通常是误把模块当层
组件	必须写「为什么这样切」；依赖只写已确认存在的
决策	必须写**放弃了什么**与代价；「先这样以后再改」尤其要留下当时的前提
开放问题	不确定的必须登记，不许用肯定语气掩盖

界面把四条产出的代码依据做成可复制的路径标签，底部是证据核验面板 （可精确定位率、ok/weak/bad 分布条、分类型明细、问题清单）。

5.3 增量审查

商用审查工具的真实工作模式是**只审 PR 改动**。本项目实现为：

解析 unified diff → 抽被改符号 → grep 反查调用点 → **合成审查简报**

关键设计：**只定位不判断**（判断归模型、判定归判官），以及 **反查改动之外的调用点** —— 这是增量审查最有价值的部分， 因为作者往往没打开那些文件。实测改动一个函数后，准确找出了未在 diff 中的调用点。

> **拿不到 diff 必须报错，不能静默退化成全仓审查。** 用户以为自己在审增量、实际审了全仓，这种偏差比报错难发现得多。

5.4 多助手隔离

「可插拔」若只是切换图标，会产生一类难发现的问题：切到架构助手，侧栏里还是审查记录。 本项目从三处隔离：

- **服务端过滤**：`/api/runs?agent=` 与 `/api/repos?agent=`，各自只返回自己的记录
- **计数同口径**：仓库卡上的运行次数与会话列表必须用同一处过滤逻辑，否则会出现「写着 5 次却点不出来」
- **正文归属**：正文区归当前选中的那一条会话独占，切换时先清空再加载，避免两次运行的内容拼接

5.5 账号与会话安全

项	实现
密码存储	scrypt（n=2^{14}, r=8, p=1）+ 每账号随机盐，存 `scrypt$n$r$p$salt$hash`
比较方式	`hmac.compare_digest` 常量时间比较，防时序攻击
登录失败	不区分「用户不存在」与「密码错误」—— 否则接口变成账号枚举器；连续失败 5 次锁 60 秒
会话	只存服务端内存，Cookie 为 HttpOnly + SameSite=Strict；**刻意不加 Secure**（本机 http 加了浏览器就不发送），注释里写明对外服务必须同时上 HTTPS
落盘	账号表原子替换写入；文件损坏时改名留档而非静默清空

5.6 前端设计系统

视觉方向是「玻璃仪表台」。核心是一条结构性规则，而非审美偏好：

材质	用在哪	为什么
玻璃 blur + 上缘内高光 + 发丝边界	顶栏、输入坞、模型菜单、弹层、运行条	这些是「浮在内容之上」的东西
实体 实心表面 + 1px 边界	发现卡、工具步、计划项、蓝图节点	这些是「内容本身」

> **为什么不能全用玻璃：**毛玻璃铺满每个卡片，层级就全平了 —— 屏幕上所有东西都在发虚光， 眼睛找不到落点。这条规则写在贡献指南里，并有测试断言守着。

配套：深色为默认、浅色是同构覆盖（不是两套样式）；字体三族分工（Sora / Instrument Sans / JetBrains Mono）；数字一律 tabular-nums；过渡只命名改变的属性，禁止 `transition: all`； 图标按钮视觉 36px、热区补到 44px。

6 测试与部署

6.1 测试体系

282 项回归测试，覆盖四类：

类别	数量	代表内容
运行时内核	约 120	主循环、上下文压缩、熔断可恢复、类型分桶、跳过与中断
工具与安全	约 45	路径逃逸、密钥拦截、非法正则、命令白名单、目录不存在
三层核验	约 35	位置核验（假发现必须拒绝、真发现不能误杀）、判官（失败保留全部）、权衡真实性
服务与前端契约	约 80	SSE 补发与终止、配置读写、前端不许有假数据、id 必须存在、玻璃只给浮层

测试密度偏高是因为大部分 bug 出在**边界条件与失败路径**上，而这两类恰恰是手工点不出来的。

> **前端契约测试的写法值得说明。**它读文件、找字符串、算对比度，看起来很笨， 但每条都对应一个真实发生过的问题：假数据回归、id 改了忘改引用、 CSS 类名与 JS 生成的类名对不上（这一条曾让蓝图连线样式整段失效）。

6.2 评估体系

6.2.1 靶场与链路

12 个靶场 → 批量运行 → 机器可读结果 → 人工复核 → **报告（脚本现算）**

12 个靶场、134 条标注。其中 8 个是**难度刻度**（s1–s8，平均 2 文件 / 62 行）， 3 个是**规模刻度**（m1–m3，1300–1800 行，含跨文件调用链与刻意埋设的干扰项）， 1 个综合样本（6 文件 / 231 行 / 23 条标注）。

> **为什么要分开两种刻度。**小样本上测出的分数会系统性偏高： 同一个综合样本，F1 从 8 个小靶场的 82% 掉到 70%。 只在小文件上验证，等于只测了最容易的场景。

6.2.2 实测结果

口径	召回率	精确率	F1
自动匹配（同文件 + 行号 ±2，8 小靶场）	86%	80%	82%
人工复核（逐条判定，带理由）	97%	95%	—
规模刻度（m1–m3，1300+ 行）	74%	72%	72%
确定性规则基线（对照）	32%	—	—

> **为什么要给两个口径。**自动匹配用「同文件 + 行号容差」判定，会产生三类系统性误判： 同一缺陷行号略有不同（被算成一漏一多）、模型指出的真问题不在答案键里（被算成误报）、 相邻缺陷互相顶替。只报对自己有利的那个口径，等于把度量方式伪装成能力差异。

6.2.3 度量本身的纪律

- **度量工具出 bug 时先怀疑度量。**本项目发生过两次：一次匹配逻辑重复消费标注， 召回被低估约 10 个百分点；一次分组口径把冗余算成误报，精确率从 100% 掉到 58%。 两次都不是模型的问题。
- **指标对行号容差高度敏感。**整体 F1 在容差 ±0 时为 61%，±4 时升到 95% —— 报单一容差下的数字没有意义。
- **单次跑分不可引用。**同一靶场三轮的召回波动可达 100/100/50。 选型类结论必须跑 3–5 次看均值与波动。
- **任何口径算出精确率或召回 > 100%，立刻停手查分母。** 本项目确实发生过一次：用宽容差统计时得出「召回 96% / 精确 110%」， 说明同一条发现被反复计入覆盖多条标注 —— 口径在自欺欺人，当场推翻改报区间。
- **标注要拿真实运行结果对照补全。**「先写标注再跑模型」会把精确率系统性低估： 同一天在三个大样本上分别低估到 30% / 22% / 31%，逐条读源码核实后修正到 83% / 56% / 77%。
- **报告不手抄。**文档里的数字由脚本从原始结果现算，避免手抄与数据脱节。

6.3 架构助手难度实测

针对架构助手单独设计了难度刻度，用三个真实开源仓库验证能力边界 （pluggy / requests / celery，规模 7k / 12k / 123k 行）。上游代码不入库， 由 `arch-cases/fetch-cases.sh` 按需克隆 —— 这是许可证边界，也是仓库体积问题。

案例	难度	规模	证据可精确定位率	轮数
pluggy	简单	36 文件 / 7.2k 行	95.3%	20（正常收尾）
requests	中等	37 文件 / 12k 行	96.9%	20
celery	困难	447 文件 / 123k 行	99.2%	23（改造前两次撞 24 轮上限）

蓝图质量评分（分层 / 组件 / 决策 / 依赖边四维 P/R/F1 加权）：简单档约 63 分、 中等档 60 分、困难档 57 分 —— **难度梯度成立。**

> **必须说清的三个局限。** ① 参考蓝图是人写的，评分上限受它约束； ② 同一案例两次跑分可能相差 16 分（79.3 vs 63.4）—— 那是模型产出在波动， 单次分数不能当「系统能力」； ③ 只测结构正确性，不测论证质量（后者由权衡检查兜，只覆盖两个硬信号）。

> **一个反直觉的发现。**初版参考蓝图只收「教科书级事实」（Session 三合一、mount 可替换）， 结果两个案例的决策维度**命中 0 条。**打开产出后看到模型给的是 「bootsteps DAG 动态装配」「公共 API 惰性模块代理」「fast_trace 快路径」—— **比参考蓝图更深入。**问题不在模型，在评分标准只认概念层、不认实现层。 合并两类条目后才反映出真实水平。

6.4 与业界水位对照

为避免「自己出题自己打分」的偏差，把结果放进公开基准的坐标系里看：

来源	F1	说明
商用产品独立排行（16,017 个真实 PR）	60.8% – 65.7%	CodeRabbit / Greptile / Copilot / Claude / Cubic 区间；厂商自测普遍虚高 15–35 个百分点
学术基准中的人类基线	0.81	所有 AI 策略的精确率-召回率曲线被人类严格支配
本项目（小靶场自动口径）	82%	缺陷明确、单次全读、自建靶场
本项目（规模刻度 1300+ 行）	72%	真实分层调用、含干扰项

> **这个对比不能简单读成「超过了商用产品」。**样本量差百倍、任务也不同 （本项目静态读全仓，商用产品审 PR 增量），而且本项目面对的是自己植入的缺陷， 人家面对的是「人类真实会评论什么」。 可信的说法是：**在「小文件、缺陷明确、单次全读」的场景下达到商用水平； 规模放大会掉分，50 文件以上的表现尚无样本，不能断言。**

6.5 部署与运行

```
# 1. 依赖（仅 httpx；开发多一个 pytest）
cd backend
pip install -r requirements-dev.txt

# 2. 配置模型：只需三项
cp .env.example .env
#   LLM_BASE_URL=https://api.deepseek.com
#   LLM_API_KEY=sk-...
#   LLM_MODEL=deepseek-chat

# 3. 启动（服务同源托管前端）
python -m harness.server --port 8765
# 打开 http://127.0.0.1:8765/

# 4. 回归
python -m pytest -q                    # 282 passed
python scripts/check_hygiene.py        # 死代码 / 未用导入

# 5. 架构案例（可选，需联网克隆上游）
bash arch-cases/fetch-cases.sh
```

项	说明
运行环境	Python 3.12+；无需 Node.js、无需数据库
数据落点	`backend/.runs/`（运行归档）· `backend/.auth/`（账号表），**均不入库**
网络	默认仅绑定本机；被审代码不出机器
离线模式	`--offline` 不调模型，产出由确定性规则给出；无 Key 时界面会自动降级并说明

7 创新点说明

7.1 技术创新点

创新点 1：三层质量约束 —— 把「可信」拆成可执行的检查

AI 审查工具最大的问题不是找不到问题，而是**找到了一堆站不住的问题。** 本项目把「可信」拆成三层按成本递增的检查：零成本的位置真实性核验、 一次模型调用的独立判官、零成本的证据与权衡审查。 每层的判据都是**确定性优先**，且每条判据都被真实数据反向修正过（详见 4.10）。

其中「判官只删不加」「判官失败保留全部」「低置信度需明确反证才允许删」三条约束， 解决的是一类很隐蔽的失败：**过滤器本身变成了故障源。**

创新点 2：助手即声明（Agent as a Declaration）

把「一个 AI 助手」抽象成一份声明：标识、系统提示、工具工厂、产出形态、强调色、只读边界与能力标记。 接入新助手只需写这份声明并注册，**工具层、渲染层、HTTP 层、前端轨道都不需要改动。** 声明是**可执行的契约** —— 声明只读的助手若注册了写工具，运行会在启动时直接失败。 前端也不按助手 id 硬编码，而是读清单里的 `outcome` 与 `capabilities` 决定渲染与入口。

创新点 3：产出形态由助手声明，渲染层注册表 + 兜底

产出不是「统一都叫发现」，而是由助手自己声明形态（发现列表 / 分层蓝图 / 其他）， 渲染层按注册表分派，未注册的形态走通用兜底。这消灭了一类很难发现的问题： **把「渲染器不认识」说成「产出为空」。**

创新点 4：缓存感知的上下文结构

静态前缀恒定不变，一切动态信息追加到轨迹末尾，使 KV 缓存命中率实测约 87%。 项目还把「前缀指纹」做成自检项，防止后续改动无意破坏这个不变量。

创新点 5：轮次预算 —— 让模型知道还剩几轮

实测发现「撞上限」的根因是模型不知道自己还剩几轮，于是边读边画直到用尽。 解决方式不是加轮数，而是每轮把剩余比例写进状态栏，并在 ≤20% 时明确要求收尾。 改造后困难案例从「两次撞 24 轮上限」变为「23 轮正常收尾」。

创新点 6：可审计的零依赖架构

后端 HTTP/SSE 层用标准库实现（外部依赖仅 httpx），前端原生 JS/CSS 无构建。 对一个**安全审计类工具**而言这不是「简陋」，而是设计选择：依赖越少越可审计、 零安装门槛、不受供应链风险影响。

创新点 7：面向代码场景的检索策略（替代向量 RAG）

代码检索的难点不是语义相似度，而是**准确的行号与结构。**项目用三条路径替代向量检索： 工具清单渐进披露、符号级读取、一次 ast 扫描产出结构清单（`sketch`）。 目标与 RAG 一致（在有限上下文里拿到需要的知识），路径更贴合代码场景。

7.2 应用创新点

创新点 8：审查与设计共用一个可插拔内核

把两类差异很大的研发任务（找缺陷 / 讲结构）放进同一工作台，共享同一套运行时、工具协议、 事件流与界面骨架，差异全部由声明表达。这验证了内核的通用性 —— 新增助手不再是「再造一个应用」，而是补一份声明。

创新点 9：架构蓝图的可评审化

架构产出不是描述性文字，而是「分层 + 组件 + 决策（含放弃的方案与代价）+ 开放问题」， 每条都带可核验的代码位置。其中**开放问题**尤其有价值：它明确标出「当前无法确定、 需要人来定」的部分，把 AI 的边界如实交付给使用者。

创新点 10：面向评审的诚实性设计

一系列刻意的「不假装」：失败必须显式；产出计数不当质量指标；无法确定的问题单列； 渲染器不认识就说认识不了；被中断的运行如实标注未完成；**界面不内置任何样例数据** —— 连不上后端时明确说未连接并给出启动命令，而不是渲染一份「看起来跑完了」的结果。

> **为什么把「诚实」算作创新点。**AI 应用最容易赢得短期好评、输掉长期信任的做法， 就是让界面看起来很完整。本项目把「不假装」做成了工程约束：界面不携带假数据、 工具失败必须上报、判官失败保留全部、指标双口径全报。这些选择让演示效果**变差**， 让可信度**变高**。

创新点 11：评估驱动开发

自带完整评估链路（靶场 → 批量运行 → 人工复核 → 报告现算），并在开发过程中 反复修正**度量本身**的错误，最终形成「双口径 + 单次不可引用 + 数字不手抄 + >100% 立即停手查分母」的纪律。项目还留下了与公开基准的对照，明确写出 「哪些结论可以下、哪些不能下」。

8 总结与展望

8.1 已交付能力

能力	状态
代码审查助手	✅ 结构化发现 + 三层核验 + 增量审查
架构设计助手	✅ 分层/组件/决策/开放问题 + 证据核验 + 权衡检查
可插拔平台	✅ 加助手 = 一份声明；前端按清单渲染
工作台界面	✅ 明暗双主题、会话隔离、实时/回放同一路径、零依赖
模型接入	✅ 三项配置接任意 OpenAI 兼容服务
测试与评估	✅ 282 项回归 + 12 靶场评估 + 双口径报告
工程规范	✅ README / 技术文档 / 贡献指南 / MIT 许可证 / 卫生检查脚本

8.2 已知边界与不足

边界	具体情况
语言覆盖	结构化分析（符号、结构清单）只做了 Python；其他语言靠文本读取
规模效应未测	最大测到 1300–1800 行；50 文件以上的表现没有样本，不能断言
波动	单次跑分波动显著（架构评分同案例可差 16 分），结论需多轮均值
运行归档未脱敏	`.runs/` 里可能含被审代码片段，尚未做凭据脱敏（已记录，未修）
HTTP 无超时兜底	畸形请求可能挂住线程（仅影响本机单用户场景）
判官覆盖有限	只对小样本几乎无收益；大样本收益明显但依赖模型质量
权衡检查是启发式	只覆盖两个硬信号，不能判断「这个决策好不好」

8.3 后续计划

1. **扩大样本**：补 500–2000 行、多语言的靶场，让规模效应可测 —— 这是当前最缺的证据
2. **多轮评测**：每档跑 3–5 次取均值与波动，替代单次跑分
3. **归档脱敏**：把 `.runs/` 里的凭据与内联密钥做正则脱敏
4. **语言扩展**：接入 Tree-sitter 之类做多语言符号提取，替代只支持 Python 的 ast

5. **判官迭代**：把判官口径与标注口径的差异做成可配置，而不是硬编码

> **一句话总结。**这个项目最有价值的不是「AI 能审代码」，而是**把 AI 的不确定性 变成了可以检查、可以讨论、可以反驳的东西** —— 每条产出指向具体行， 每条决策写出代价，每个不确定标为开放问题，每个指标给出双口径。 在这个前提下，「AI 辅助研发」才是一个工程问题，而不是一个期待。